



Laboratorio 3: Sistemas Distribuidos

DiscoPass

Profesor: Jorge Díaz

Ayudantes de Lab: Felipe Marchant V. & Nelson Sepúlveda V.

Junio 2025

1 Objetivos del Laboratorio

- Implementar y diferenciar múltiples **modelos de consistencia** (eventual y centrada en el usuario) dentro de un mismo sistema.
- Diseñar un sistema de **manejo de sesiones** para asegurar las diferentes consistencias.
- Consolidar el uso de Go, gRPC y Docker para la construcción de sistemas distribuidos complejos y tolerantes a fallos.

2 Tecnologías

- El lenguaje de programación a utilizar es **Go**.
- Para la comunicación síncrona se utilizará **gRPC**.
- Para la definición de mensajes síncronos se usará **ProtocolBuffers**
- Para distribuir el programa se utilizará **Docker**

3 Introducción

La logística de entrega de comida a domicilio (delivery) constituye uno de los desafíos más dinámicos y demandantes en el desarrollo de aplicaciones modernas. Durante las “horas punta”, miles de clientes, restaurantes y repartidores interactúan simultáneamente con la plataforma, generando un flujo masivo y constante de nuevos pedidos y actualizaciones de estado.

En este entorno de alta concurrencia, la confiabilidad del sistema depende de equilibrar distintas necesidades de consistencia. Por un lado, cuando un usuario confirma y paga su carrito de compras, requiere certeza absoluta e inmediata de que su pedido fue procesado correctamente. Por otro lado, el seguimiento en tiempo real del progreso de la entrega requiere una distribución eficiente de los

datos a lo largo de toda la red, priorizando la disponibilidad sobre la sincronización instantánea global.

En este laboratorio, se simulará la arquitectura de “DistriEats”, un ecosistema distribuido de gestión logística compuesto por:

- **Cientes Hambrientos (Usuarios):** Actores que realizan pedidos en la aplicación y requieren leer su propio estado actualizado inmediatamente tras una operación de escritura (garantía Read Your Writes).
- **Gateway de Pedidos (Coordinador):** Un intermediario encargado de interceptar las peticiones de los clientes y gestionar la afinidad de sesión para asegurar una experiencia de usuario coherente en sus primeras interacciones.
- **Broker de Logística Central:** Un enrutador sin estado que recibe las actualizaciones de eventos y distribuye la carga operativa hacia la capa de almacenamiento.
- **Servidores de Zona (Datanodes):** Un sistema de base de datos distribuido y replicado. Son los encargados de almacenar el estado de los pedidos, comunicándose entre pares mediante un protocolo asíncrono (gossip) y resolviendo actualizaciones concurrentes utilizando relojes vectoriales.
- **Restaurantes y Repartidores (Simulados):** Entidades que actúan como emisores constantes de eventos, reportando los cambios en el ciclo de vida de cada entrega (ej. Preparando, En Camino, Entregado).

El sistema diseñado debe ser tolerante a fallos y orientado a la convergencia. La caída temporal de un nodo de la base de datos o el desorden en la llegada de los mensajes de red no debe detener la operación de la plataforma, ni provocar que un pedido retroceda erróneamente a un estado anterior. El desafío central será garantizar que toda la red converja de forma eventualmente consistente hacia la verdad absoluta de cada entrega.

4 Laboratorio

4.1 Contexto

En la industria de la entrega de comida a domicilio (delivery), la gestión logística se ha convertido en un proceso de ritmo vertiginoso y alta criticidad. Durante los horarios de mayor demanda, como los almuerzos corporativos o las noches de fin de semana, miles de usuarios pueden conectarse simultáneamente para realizar pedidos, concretar pagos y monitorear el progreso de su entrega con alta expectativa.

En este escenario de alta concurrencia, la red de suministro (restaurantes y repartidores) emite un flujo masivo y constante de eventos para actualizar el estado de cada pedido a medida que avanza desde la cocina hasta el domicilio. Al mismo tiempo, la plataforma central enfrenta el desafío de procesar estas solicitudes en tiempo real, garantizando que los clientes vean la confirmación inmediata de sus transacciones, mientras el sistema descentralizado resuelve los retrasos de red y posibles conflictos para asegurar que la trazabilidad de un pedido evolucione de manera coherente sin perder información en el proceso.

4.2 Explicación

Para este laboratorio, deberán simular un ecosistema distribuido de logística y seguimiento de pedidos operando para cuatro cadenas de comida rápida ficticias:

- BurgerNode
- DockerPizza
- SushiStream
- GopherTacos

Cada uno de estos restaurantes y sus respectivos repartidores actuarán como **productores de eventos**, encargados de emitir actualizaciones constantes sobre el ciclo de vida de los pedidos (por ejemplo: Recibido, Preparando, En Camino, Entregado, Cancelado). Estas actualizaciones serán enviadas hacia un **Broker Central**, el cual actuará como un orquestador que enruta las solicitudes sin mantener estado interno de la lógica de negocio.

El Broker Central distribuirá de forma asíncrona la información de los eventos hacia un sistema de **base de datos distribuida** compuesto por múltiples servidores conocidos como **Datanodes**. Este subsistema de almacenamiento operará estrictamente bajo un modelo de **consistencia eventual**. Para asegurar la convergencia global de la información, los Datanodes propagarán las actualizaciones mediante un mecanismo de comunicación entre pares (gossip) y detectarán conflictos empleando **relojes vectoriales** combinados con políticas de resolución deterministas.

Por otra parte, los **Clientes Hambrientos (Usuarios)** interactuarán con la plataforma realizando operaciones transaccionales críticas, como efectuar un nuevo pedido. Para satisfacer la necesidad de estos usuarios de obtener confirmación inmediata de sus acciones, todas sus peticiones pasarán primero por un **Gateway de Pedidos (Coordinador)**. La misión exclusiva de este Coordinador será garantizar el modelo de consistencia Read Your Writes, gestionando la afinidad de sesión en memoria para asegurar que las lecturas de un cliente sean redirigidas al mismo Datanode que procesó su escritura más reciente.

Durante la simulación, deberán considerar y manejar **fallos y anomalías de red** inevitables:

- La caída temporal de un Datanode, el cual deberá ser capaz de resincronizarse con sus pares al volver a estar en línea.
- La llegada de eventos desordenados o actualizaciones concurrentes emitidas por el Broker Central hacia distintos Datanodes, requiriendo el uso mandatorio de los relojes vectoriales para establecer relaciones causales.
- Interacciones de usuarios que exigen ver sus pedidos inmediatamente consolidados frente a una base de datos que aún no propaga los cambios a todos sus nodos.

El reto fundamental de este laboratorio es construir una arquitectura robusta que logre abstraer la complejidad de la red. Deberán demostrar que el sistema satisface la inmediatez requerida por el usuario (RYW) mientras tolera caídas y demoras, garantizando que todos los nodos converjan eventualmente hacia la verdad absoluta e histórica de cada entrega en la red.

4.3 Clientes Hambrientos (Clientes Read Your Writes)

Estos clientes representan a los usuarios finales de la aplicación “DistriEats” que interactúan con el sistema para realizar una operación transaccional crítica, como la confirmación y pago de un pedido de comida. Su principal requerimiento es la certeza de que sus acciones se reflejan de manera inmediata y correcta en su propia vista de la aplicación al momento de revisar su orden.

Misión Principal:

Realizar una operación de escritura (confirmación de un pedido) y verificar su resultado de forma inmediata, garantizando que el sistema respeta la consistencia centrada en el usuario (Read Your Writes).

Responsabilidades clave:

- **Solicitud de Estado Inicial:** Antes de una escritura, el cliente debe realizar una operación de lectura al Gateway de Pedidos (Coordinador) para obtener el estado actual de los recursos (por ejemplo, el menú y la disponibilidad de productos en el restaurante seleccionado).
- **Envío de Operación de Escritura:** Construir y enviar la orden de compra al Gateway de Pedidos. Para garantizar la idempotencia, cada solicitud de pedido debe incluir un identificador de solicitud único que permita al sistema descartar reintentos duplicados en caso de inestabilidad en la red.
- **Lectura de Confirmación:** Inmediatamente después de recibir una confirmación de que la escritura (el pedido) fue exitosa, el cliente debe ejecutar una solicitud de lectura para obtener el estado actualizado de su entrega.
- **Validación de Consistencia:** Es responsabilidad del cliente validar que la respuesta a esta lectura de confirmación contiene efectivamente los datos de su pedido recién creado. Esta validación confirma que la garantía Read Your Writes se ha cumplido exitosamente en el sistema.

Consideraciones de Implementación:

- El cliente debe ser lo suficientemente “stateful” (con estado) como para recordar en memoria el identificador único de su solicitud y los datos del pedido que envió, de modo que pueda realizar la validación de consistencia posterior.
- Debe estar programado para manejar posibles errores devueltos por el Gateway de Pedidos, como la situación donde un producto del menú se agotó justo antes de procesar el pago (condición de carrera).
- Se requerirá implementar múltiples instancias de este cliente (por ejemplo, 3 clientes operando simultáneamente) para simular el tráfico en la plataforma.

Consideraciones de Implementación:

- Se comunica de manera exclusiva y directa con el Gateway de Pedidos (Coordinador). No tiene conocimiento ni acceso directo al Broker Central o a los Datanodes.

4.4 Gateway de Pedidos (Coordinador)

El Gateway de Pedidos es un componente de infraestructura crítico que actúa como un proxy con estado de sesión. Su única misión es implementar la garantía Read Your Writes para los Clientes Hambrientos, separando esta lógica de enrutamiento de la lógica de distribución del Broker Central.

Misión Principal:

Garantizar la consistencia Read Your Writes actuando como un intermediario con estado de sesión entre los Clientes Hambrientos y el sistema principal.

Responsabilidades Clave:

- **Gestión de Afinidad de Sesión (Sticky Sessions):** Mantener un mapeo en memoria que asocie a un cliente específico con el Datanode que procesó su última operación de escritura (su pedido más reciente). Este mapeo debe tener un tiempo de vida (TTL) configurado para evitar que el uso de memoria crezca indefinidamente con el paso del tiempo.
- **Redirección de Escritura:** Al recibir una solicitud de escritura, el Gateway debe seleccionar un Datanode disponible (o solicitarle la asignación de uno al Broker Central), registrar esta afinidad en su mapeo de sesión local, y luego reenviar la solicitud al Broker Central para su procesamiento en la red.
- **Intercepción y Redirección de Lectura:** Al recibir una consulta de lectura por parte de un cliente (por ejemplo, recargar la pantalla para revisar el estado de su orden), el Gateway debe verificar si existe una sesión activa para ese cliente en su mapeo. Si la sesión existe, debe reenviar la solicitud directamente al Datanode asociado; si no existe una sesión previa, simplemente la reenvía al Broker Central para que este la balancee de forma normal.

Consideraciones de Implementación:

- El Gateway de Pedidos NO debe incluir ninguna lógica de negocio relacionada con menús, restaurantes o validaciones de delivery; su única función es enrutar solicitudes basándose en el estado de las sesiones.
- Aunque en un sistema de producción real este componente estaría replicado para garantizar alta disponibilidad, para los fines de este laboratorio se deberá implementar como una única instancia (Se requiere implementar 1).

Interacciones Principales:

- Recibe solicitudes de lectura y escritura exclusivamente de los Clientes Hambrientos.
- Envía solicitudes hacia el Broker Central, y se comunica directamente con los Datanodes cuando necesita resolver lecturas que poseen afinidad de sesión activa.

4.5 Broker de Logística (Broker Central)

El Broker de Logística es el núcleo orquestador del ecosistema. Actúa como una fachada que enruta las solicitudes de manera eficiente y desacopla a los clientes (y al Gateway) de la complejidad interna de los servicios de almacenamiento.

Misión Principal:

Enrutar eficientemente todas las solicitudes a los subsistemas correctos, actuando como un director de tráfico sin estado para la lógica interna y distribuyendo las constantes actualizaciones logísticas de la red.

Responsabilidades Clave:

- **Distribución de Actualizaciones (Broadcast):** Recibir las constantes actualizaciones sobre el estado de los pedidos (provenientes de los restaurantes y repartidores simulados) y distribuirlas de forma asíncrona a los Datanodes para su almacenamiento.
- **Balanceo de Carga:** Recibir las consultas y operaciones de escritura provenientes del Gateway de Pedidos (aquellas que no requieren validación de afinidad directa) y reenviarlas a un Datanode disponible utilizando una estrategia equitativa, como Round Robin.

Consideraciones de Implementación:

- El Broker debe ser lo más liviano y sin estado posible. Su verdadero valor dentro de la arquitectura reside en su capacidad de enrutamiento y distribución ágil, no en el procesamiento o el almacenamiento de datos.
- No tiene conocimiento ni responsabilidad sobre la lógica de Read Your Writes. Simplemente procesa de manera transparente las solicitudes que le llegan desde el Gateway de Pedidos y las deriva.
- Para simular la emisión constante de actualizaciones de estado por parte de la red de delivery, el Broker obtendrá los eventos logísticos desde un archivo .csv (la implementación técnica de esto será explicada en la ayudantía correspondiente).
- Al igual que el Gateway, se debe implementar como una única instancia central (Implementar 1).

Interacciones Principales:

- Recibe solicitudes enviadas por el Gateway de Pedidos (Coordinador).
- Envía solicitudes operativas y los eventos logísticos (leídos del archivo) hacia los Datanodes de la red.

4.6 Servidores de Zona (Datanodes)

Los Servidores de Zona conforman la columna vertebral del almacenamiento de “DistriEats”. Son los nodos de la base de datos distribuida y actúan como la fuente de verdad (eventualmente consistente) para el estado de cada pedido en la red. A diferencia del Gateway o el Broker, estos nodos retienen los datos históricos y actuales de las entregas.

Misión Principal:

Almacenar y servir los datos del ciclo de vida de los pedidos de forma replicada y tolerante a fallos, utilizando relojes vectoriales y comunicación descentralizada para converger pacientemente hacia un estado globalmente consistente.

Responsabilidades Clave:

- **Almacenamiento y Lógica Causal:** Guardar en memoria o en disco el estado de los pedidos (ej. Preparando, En Camino). Cada vez que un Datanode procesa una actualización proveniente del Broker o de otro nodo, debe fusionar (merge) el reloj vectorial adjunto al evento con su propio reloj local antes de aplicar cualquier cambio.
- **Detección y Resolución de Conflictos:** Al recibir una actualización para un pedido existente, el nodo debe comparar el reloj vectorial entrante con la versión que tiene almacenada. Si los relojes demuestran que no existe una relación causal (es decir, las actualizaciones son concurrentes y una no precede a la otra), se ha detectado un conflicto. El Datanode debe aplicar una política de resolución determinista. En el contexto de DistriEats, esto significa que el estado más avanzado lógicamente en la cadena de entrega siempre gana (ej. Entregado sobrescribe a En Camino de forma automática, y Cancelado tiene prioridad absoluta sobre cualquier otro estado).
- **Sincronización Entre Pares (Protocolo Gossip):** Periódicamente, cada Datanode debe iniciar una comunicación con otro Datanode (elegido al azar dentro del clúster) para intercambiar el estado de los pedidos. Este mecanismo permite propagar la información faltante, superando las interrupciones y obstáculos de la red hasta consolidar una visión coherente y actualizada en todos los nodos.

Consideraciones de Implementación:

- Los Datanodes son completamente agnósticos a los conceptos de “usuario”, “interfaz” o “sesión”. Su mundo se rige exclusivamente por identificadores de pedido (claves), estados logísticos (valores) y metadatos de causalidad (relojes vectoriales).
- Deben ser altamente resilientes. La red puede entregarles mensajes duplicados o en un orden incorrecto; su lógica de relojes vectoriales debe garantizar que un pedido nunca “retroceda” a un estado anterior debido a un paquete de red atrasado.
- Se requiere implementar múltiples instancias de esta entidad (por ejemplo, 3 Datanodes) para construir un clúster funcional y poner a prueba la convergencia eventual.

Interacciones Principales:

- Reciben solicitudes de lectura y actualizaciones de escritura reenviadas por el Broker Central.

- Reciben lecturas directas del Gateway de Pedidos (cuando existe afinidad de sesión por Read Your Writes).
- Se comunican de manera intensiva entre sí (con los otros Datanodes) para ejecutar el protocolo de sincronización gossip.

4.7 Restaurantes y Repartidores (Productores de Eventos Simulados)

Esta entidad representa a los actores externos en el mundo real que alimentan constantemente el sistema con información operativa. A diferencia de los clientes que leen y escriben mediante interfaces, los restaurantes y repartidores actúan únicamente como un motor incesante de actualizaciones de estado que fluyen hacia la plataforma.

Misión Principal:

Generar y emitir un flujo constante, asíncrono y realista de eventos logísticos hacia la red, simulando el avance natural de las entregas de comida.

Responsabilidades Clave:

- **Emisión de Ciclo de Vida:** Notificar al sistema central cada vez que un pedido avanza a su siguiente etapa (por ejemplo, de Preparando a En Camino).
- **Gestión Inicial de Relojes Vectoriales:** Cada actualización de estado emitida debe llevar asociado un reloj vectorial actualizado. Esto es fundamental para que, una vez que el mensaje viaje por la red, los Datanodes tengan el contexto temporal necesario para resolver posibles colisiones o desórdenes.

Consideraciones de Implementación:

- Para facilitar la simulación y evitar el desarrollo de interfaces complejas, la emisión constante de actualizaciones de estado se obtendrá leyendo los eventos desde un archivo .csv estructurado.
- El sistema debe simular la llegada de estos eventos de forma realista, lo que implica enviar un nuevo evento con intervalos de tiempo aleatorios (por ejemplo, cada 1 a 3 segundos).
- Esta entidad no requiere ser un contenedor pesado con estado, sino más bien un script o proceso ligero adjunto al Broker que iterativamente lee el archivo y “dispara” los eventos hacia la red.

Interacciones Principales:

- Se comunica de manera unidireccional con el Broker Central, enviándole el flujo de actualizaciones continuas para que este las distribuya a los Datanodes.

4.8 Modelos de Consistencia a Implementar

La correcta implementación de los modelos de consistencia es crucial para el funcionamiento predecible y correcto de la plataforma. A continuación, se detalla cómo deben aplicarse estos conceptos en el contexto logístico de “DistriEats”.

4.8.1 Consistencia Eventual con Relojes Vectoriales

Definición Aplicada:

Las operaciones que modifican el estado de las entregas (ej. cambios de Preparando a En Camino) no se aplican en un orden total global instantáneo. Cada Datanode mantiene su propia vista local del estado. Para garantizar que todos los nodos converjan eventualmente a una visión idéntica y causalmente correcta del ciclo de vida de cada pedido, se utilizarán relojes vectoriales.

Mecánica de los Relojes Vectoriales:

- Cada entidad que origina cambios de estado (representada por la lectura del archivo en el Broker) y cada Datanode mantendrá un reloj vectorial. Para simplificar, podemos asumir que cada Datanode tiene una entrada asignada en el vector.
- Cada vez que el Broker emite una actualización de pedido, adjunta el reloj vectorial actualizado en el mensaje gRPC.
- Cuando un Datanode recibe un mensaje, actualiza su propio reloj tomando el componente máximo por posición (merge) con el reloj del mensaje. Antes de aplicar el cambio de estado en la base de datos, debe comparar los relojes para detectar si la actualización es causalmente posterior, anterior o concurrente a la versión que ya tiene almacenada.
- Regla de Resolución Determinista: En caso de un conflicto (dos actualizaciones concurrentes para el mismo pedido que llegan desordenadas), el Datanode debe aplicar una política estricta basada en el avance lógico del delivery. El orden de prioridad es: Recibido → Preparando → En Camino → Entregado (Es decir, el estado más avanzado es el que manda). El estado Cancelado tiene prioridad absoluta sobre todos los demás.

Escenario Clave:

1. El Broker emite una actualización para el Pedido #1042: estado = “En Camino”. El mensaje lleva un reloj vectorial.
2. Casi simultáneamente, debido a un retraso de red previo, llega otra actualización para el mismo pedido: estado = “Preparando”, con otro reloj vectorial.
3. Un Datanode recibe ambos mensajes desordenados. Al comparar los relojes vectoriales, determina que los eventos son concurrentes (no hay una relación causal directa entre ellos según el vector).
4. El Datanode aplica la política de resolución: como “En Camino” es un estado lógicamente más avanzado que “Preparando”, consolida el estado final como “En Camino”.
5. El Datanode fusiona ambos relojes con el suyo y propaga esta versión consolidada a otros nodos mediante el protocolo gossip, ayudando al clúster a converger.

4.8.2 Read Your Writes (Clientes Hambrientos)

Definición Aplicada:

Después de que un Cliente Hambriento realiza una operación de escritura que modifica su propio estado (ej. la creación y pago de un pedido), cualquier consulta subsecuente que realice a través del Gateway de Pedidos debe devolver un estado que refleje esa modificación de forma inmediata.

Escenario Clave:

1. Un Cliente (Usuario 1) envía una solicitud de CrearPedido al Gateway, confirmando su carrito de compras (ID Único: Req-99).
2. El Gateway procesa la solicitud, selecciona el Datanode 2, reenvía la escritura y confirma la operación al Usuario 1.
3. Inmediatamente después, el Usuario 1 recarga su aplicación, enviando una solicitud ConsultarEstado al Gateway.
4. La respuesta debe indicar que su pedido existe y está en estado “Recibido”. No debe, bajo ninguna circunstancia, mostrar que su historial está vacío debido a que la consulta cayó en un Datanode (ej. Datanode 1) que aún no ha sido sincronizado por gossip.

Implementación Exigida:

Esta garantía debe implementarse estrictamente a través de la afinidad de sesión gestionada por el Gateway de Pedidos. Al procesar la escritura del Usuario 1, el Gateway registra en memoria a qué Datanode se envió la solicitud. Las lecturas posteriores de este usuario son dirigidas forzosamente a ese mismo Datanode, garantizando que leerá el dato exacto que acaba de escribir.

4.9 Fases de la Ejecución

Para asegurar un desarrollo ordenado y facilitar la evaluación del laboratorio, la simulación del clúster de “DistriEats” debe estructurarse y evidenciarse en las siguientes fases lógicas de ejecución:

4.9.1 Fase 1: Inicialización y Descubrimiento

- **Despliegue:** Todos los contenedores (Datanodes, Broker, Gateway, Clientes) se instancian y levantan mediante el archivo Makefile provisto.
- **Conexiones Base:** Los Datanodes inician sus servidores gRPC y quedan a la escucha. El Broker Central y el Gateway de Pedidos establecen sus conexiones iniciales con los Datanodes disponibles en el clúster.

4.9.2 Fase 2: Apertura de la Red Logística

- **Emisión de Eventos:** El proceso encargado de simular a los restaurantes y repartidores comienza a leer secuencialmente el archivo pedidos.csv. Inicia el envío de un flujo constante de eventos de actualización (por ejemplo, cada 1 a 3 segundos) hacia el Broker Central.
- **Distribución:** El Broker reenvía estos eventos a los Datanodes, utilizando una política equitativa (ej. Round Robin).

- **Gossip Activo:** En paralelo a la llegada de mensajes, los Datanodes inician sus rutinas en segundo plano (background goroutines). Cada Datanode contactará a un par aleatorio cada cierto intervalo de tiempo (ej. cada 5 segundos) para intercambiar su estado local y aplicar la fusión de relojes vectoriales.

4.9.3 Fase 3: Concurrencia Transaccional (Read Your Writes)

- **Pedidos de Clientes:** Mientras la red logística procesa el intenso tráfico de actualizaciones de entrega, los contenedores de los Clientes Hambrientos “despiertan” y envían solicitudes transaccionales (creación de nuevos pedidos) al Gateway de Pedidos.
- **Validación de Sesión:** El Gateway enruta la escritura a un Datanode y guarda la afinidad de la sesión en memoria. Inmediatamente después de recibir la confirmación de la escritura, el Cliente lanza una consulta de estado. El Gateway intercepta esta lectura y la redirige forzosamente al nodo afín.
- **Evidencia:** El Cliente debe imprimir en su consola un mensaje de éxito validando que su pedido recién creado fue encontrado en la lectura subsecuente, demostrando la garantía RYW.

4.9.4 Fase 4: Simulación de Caos y Resiliencia (Tolerancia a Fallos)

- **Caída Inducida:** Durante el clímax de la simulación (mientras se procesan compras y eventos de red), se debe detener manualmente (ej. docker stop) o de forma programada uno de los Datanodes.
- **Operatividad:** El ecosistema no debe colapsar. El Broker y el Gateway deben manejar el error de conexión y enrutar las nuevas peticiones hacia los Datanodes sobrevivientes.
- **Recuperación:** Al reiniciar el Datanode caído (docker start), este debe arrancar con su información persistida (o vacía, dependiendo de la implementación) y utilizar de inmediato el protocolo gossip para solicitar la información histórica a sus pares, hasta alcanzar el mismo estado del clúster.

4.9.5 Fase 5: Convergencia Global y Cierre

- **Tiempo de Gracia:** Una vez que el archivo .csv finaliza su lectura y los Clientes terminan sus transacciones, se otorgará un tiempo de ventana (ej. 15 segundos) para que la red se estabilice y el protocolo gossip propague las últimas diferencias entre los nodos.
- **Auditoría Final:** Transcurrido el tiempo de gracia, cada Datanode debe imprimir su estado final (el registro completo de todos los pedidos y sus estados lógicos finales) en un archivo de log local. Para demostrar el éxito de la consistencia eventual, el contenido de todos los archivos generados por los Datanodes activos debe ser exactamente igual.

4.10 Reporte Final

Al finalizar la simulación y transcurrido el tiempo de gracia para la convergencia, el sistema (ya sea el Broker Central o un script de auditoría) debe generar un archivo consolidado llamado Reporte.txt. El objetivo de este documento es evidenciar de manera clara que el clúster alcanzó un estado global consistente y registrar el éxito de las operaciones centradas en el usuario.

Este reporte servirá como prueba fundamental para la evaluación de su laboratorio, facilitando la revisión de los resultados obtenidos tras el caos de la simulación.

Contenido esperado:

1. **Resumen Global de Pedidos:** El estado consolidado final de cada pedido que ingresó al sistema, demostrando que la política de resolución de conflictos funcionó y no existen divergencias entre los Datanodes.
2. **Registro de Validaciones (RYW):** Un listado de las validaciones exitosas realizadas por los Clientes Hambrientos, demostrando que el Gateway enrutó correctamente las lecturas.

Ejemplo de formato para Reporte.txt:

Reporte.txt

```
=== REPORTE FINAL: DISTRIEATS ===

[ESTADO GLOBAL DE PEDIDOS - Convergencia Alcanzada]
Pedido ID: Ped-001 | Estado Final: Entregado | Reloj Vectorial: [DN1:3, DN2:2, DN3:2]
Pedido ID: Ped-002 | Estado Final: Cancelado | Reloj Vectorial: [DN1:1, DN2:1, DN3:3]
Pedido ID: Ped-003 | Estado Final: En Camino | Reloj Vectorial: [DN1:2, DN2:4, DN3:2]

[AUDITORIA READ YOUR WRITES]
- Cliente 1 (Ped-004): Validacion Exitosa en Datanode 2 (Afinidad de sesion confirmada).
- Cliente 2 (Ped-005): Validacion Exitosa en Datanode 1 (Afinidad de sesion confirmada).
=====
```

5 Uso de Docker

Todos los procesos del laboratorio deben ejecutarse en contenedores Docker dentro de las máquinas virtuales. Todas las entidades deben estar aisladas en contenedores separados.

Ejemplo de distribución de las entidades en las máquinas virtuales:

- MV1: Broker Central / Productor de Eventos
- MV2: Gateway de Pedidos / Cliente Hambriento 1 / Datanode 1
- MV3: Cliente Hambriento 2 / Datanode 2
- MV4: Cliente Hambriento 3 / Datanode 3

6 Restricciones

Las librerías de Golang permitidas son:

- `fmt`
- `log`
- `errors`
- `os`
- `io`
- `bufio`
- `time`
- `sync`
- `sort`
- `math/rand`
- `encoding/csv`
- `encoding/json`
- `net`
- `context`
- `flag`
- `strconv`
- `strings`
- `grpc`

Además de las necesarias para el funcionamiento de gRPC y Protocol Buffers. Todo uso de librerías externas que no se han mencionado en el enunciado debe ser consultado con los ayudantes.

7 Consideraciones

Esta sección detalla requerimientos técnicos específicos y aclara elementos de la simulación que deben ser respetados estrictamente durante el desarrollo de “DistriEats”:

- **Comunicación y Arquitectura:**

- Todas las comunicaciones entre entidades deben realizarse estrictamente mediante **gRPC** y **Protocol Buffers**.
- No está permitido el uso de colas de mensajes externas como RabbitMQ, Kafka, NATS, etc.

- **Datanodes (Consistencia Eventual):**

- La propagación de actualizaciones asíncronas entre los Datanodes debe implementarse mediante un mecanismo de comunicación entre pares (gossip).
- Cada actualización de estado de un pedido que fluya por la red debe llevar asociado un reloj vectorial.
- Deben ser capaces de detectar y resolver conflictos de escritura basándose en los relojes vectoriales y la regla de resolución determinista explicada en la Sección 4.8.1.
- En caso de la caída temporal de un nodo, este debe ser capaz de resincronizarse con sus pares al volver a estar en línea para alcanzar la consistencia eventual global.

- **Clientes y Gateway (Read Your Writes):**

- La lógica para garantizar Read Your Writes debe residir en el Gateway de Pedidos (Coordinador), principalmente a través de la gestión de afinidad de sesión en memoria.
- Todas las solicitudes de creación de pedidos (escritura) de los Clientes Hambrientos deben ser idempotentes, incluyendo un ID único para evitar duplicidad si el cliente reintenta la conexión.

- **Simulación y Entorno:**

- El sistema debe simular la llegada de actualizaciones de los pedidos de forma continua y realista (ej. un nuevo evento leído del archivo cada 1 a 3 segundos con intervalos aleatorios).
- Las entidades deben generar logs claros en sus respectivas consolas que permitan trazar la ejecución, incluyendo la recepción de mensajes, la detección y resolución de conflictos mediante vectores, y las redirecciones de sesión del Gateway.
- El Broker Central (o un script designado) debe generar el archivo Reporte.txt al finalizar el tiempo de convergencia, resumiendo el estado final de los pedidos y las lecturas exitosas.
- Cada entidad debe ejecutarse en un contenedor Docker independiente. Se debe incluir un Makefile que facilite la ejecución de las distintas configuraciones de las máquinas virtuales (ej. make docker-VM1).

- **Archivos de Configuración:**

- Los archivos necesarios serán entregados por los ayudantes. A la hora de evaluar se usará una configuración distinta así que nada debe estar “hardcodeado”.
- El único archivo proporcionado por el equipo de ayudantes es “pedidos.csv”, el cual es el archivo con el flujo de eventos logísticos a simular (ID del pedido, estado, tiempo relativo, etc.).

- **Contenedores Docker:**

- Cada entidad debe ejecutarse en un contenedor Docker independiente.
- Se debe incluir un `Makefile` con atajos como:
 - * `make docker-VM1`
 - * `make docker-VM2`
 - * `make docker-VM3`
 - * `make docker-VM4`

- **Consultas y Soporte:**

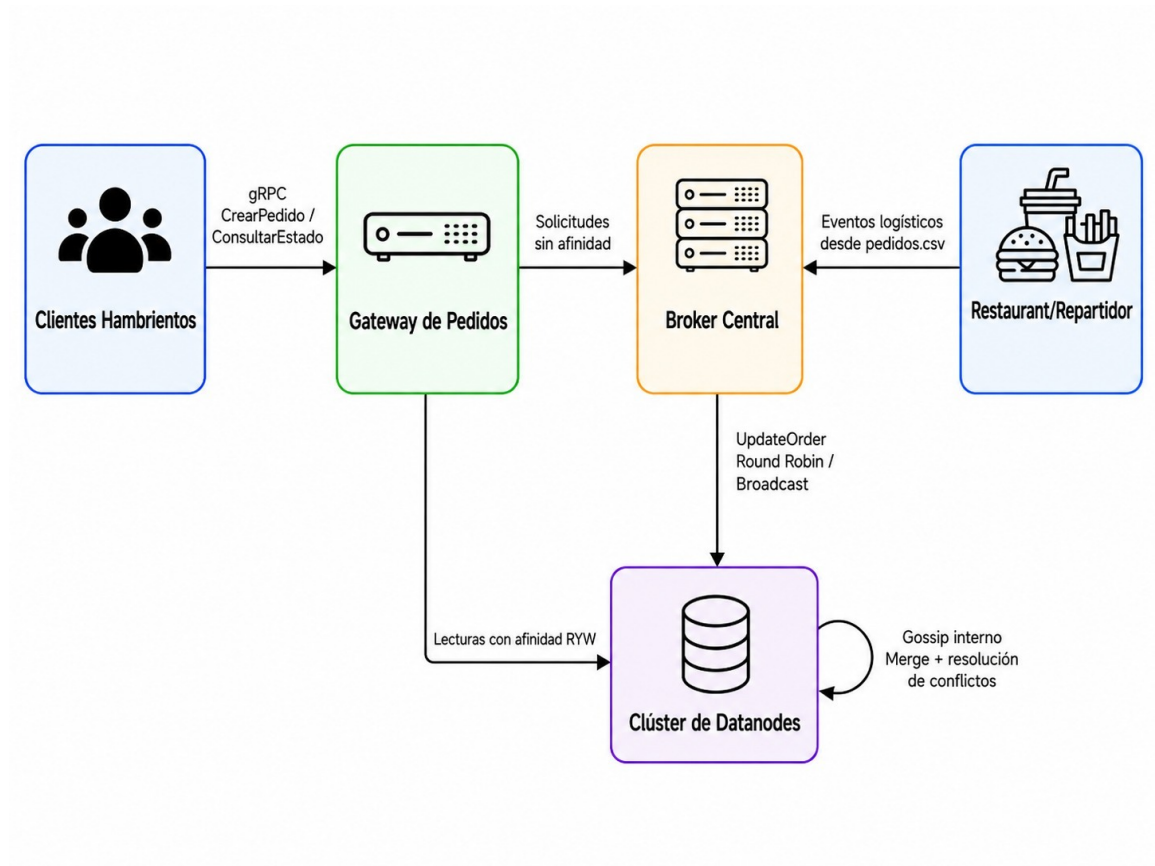
- Se realizará una ayudantía para detallar el enunciado y resolver dudas generales.
- Las consultas deben realizarse en el foro habilitado en Aula o al correo de los ayudantes:
 - * `felipe.marchantv@usm.cl`
 - * `nelson.sepulveda @usm.cl`
- No se responderán consultas enviadas con menos de **48 horas antes de la fecha de entrega original**. En caso de extensión de plazo, para esta regla se considerará la fecha original de entrega.

8 Reglas de Entrega

- La tarea se entrega en grupos de 2 a 3 personas previamente asignados en aula.
- La fecha de entrega es el día **1 de julio 23:59 hrs.**
- La tarea se revisará en las máquinas virtuales, por lo que los archivos necesarios para la correcta ejecución de esta deben estar en ellas. Recuerde que el código debe estar indentado, comentado, sin warnings y sin errores.
- Además de los códigos en las máquinas virtuales y github, deberán subir un archivo comprimido que contenga todos los códigos desarrollados en carpetas separadas por entidad en formato **.zip** con el nombre **GrupoXX-LabY.zip**. Donde XX es el número de su grupo e Y es el número del laboratorio. Ejemplo: *Grupo07-Lab3.zip*.
- Debe dejar un **MAKEFILE** o similar en cada máquina virtual asignada a su grupo para la ejecución de cada entidad.
- Debe dejar un **README** en la entrega asignada a su grupo con nombre y rol de cada integrante, además de la información necesaria para ejecutar los archivos.
- El uso de **Docker** es **obligatorio**. Teniendo nota 0 aquellas implementaciones que no hagan uso de Docker.
- Las faltas de README y/o copias serán evaluadas con nota 0.
- Todos los descuentos y aclaraciones necesarias están disponibles en el documento “**Reglas de Laboratorio**” disponible en el Aula virtual. Favor revisarlo con prontitud para evitar inconvenientes.

9 Anexos

9.1 Entidades



9.2 Diagramas de Secuencia

